PBCP: Pre-Billing Cost Prevention for Intent-Aware Cloud Governance

Keerthi Rapolu
Sreeja Katta

Abstract

Cloud governance systems usually detect waste only after resources have been provisioned, used, and billed. This timing makes many optimizations economically reactive even when post-execution diagnosis is accurate. We present *Pre-Billing Cost Prevention* (PBCP), an intent-aware governance framework that infers workload intent from natural-language submissions, retrieves similar historical workloads, simulates likely utilization and cost before execution, and applies decision-theoretic intervention before waste is incurred. PBCP combines pre-billing prevention with runtime governance and policy learning, and it evaluates system behavior using Cost Prevention Score (CPS), Execution Success Rate (ESR), and Intent-Fit Score (IFS). Across a controlled benchmark of 500 workloads and 28,423 runs, PBCP achieves utilization MAE of 0.054 in simulation calibration, improves IFS-based anomaly-detection F1 to 0.7608 versus 0.6054 for a CPU-threshold baseline, and reaches peak CPS of 0.733, a 56× improvement over the No Phase 3 baseline. These results suggest that cloud governance benefits from moving intervention earlier in the execution lifecycle rather than relying on post-billing recommendation alone.

1 Introduction

A data engineering team provisions a 20-node Spark cluster for a nightly ETL workload expected to complete in two hours. The workload finishes in 37 minutes, but the cluster remains active overnight, accumulating hundreds of dollars in idle compute charges. Hours later, a cloud optimization advisor flags the cluster as underutilized. The waste has already occurred.

This failure is not primarily a monitoring problem. It is a governance-timing problem. Existing cloud governance systems predominantly detect inefficiency after resources have already been provisioned, used, and billed. They optimize after execution, reason from telemetry after the fact, and react only after cost has already been incurred. Even accurate post-execution recommendations arrive too late to prevent the original waste.

The core insight of this paper is that cloud waste is often caused by divergence between *declared workload intent* and *actual runtime behavior*. A workload submitted as a large, recurring ETL job may in practice behave like a short-lived, lightly utilized batch task. If intent can be inferred early, and if likely utilization and cost can be simulated before provisioning, governance can intervene before the waste is created.

We present *Pre-Billing Cost Prevention* (PBCP), an intent-aware cloud governance framework that moves the intervention point to the left of billing. PBCP infers workload intent from natural-language descriptions, retrieves similar historical workloads, predicts likely resource divergence through pre-execution simulation, and applies decision-theoretic interventions such as `BLOCK`, `AUTO_CORRECT`, `SUGGEST`, or `PASS` before or during execution.

PBCP is evaluated with three complementary metrics. *Cost Prevention Score* (CPS) measures how much waste was prevented before billing, *Execution Success Rate* (ESR) measures whether intervention preserved useful execution, and *Intent-Fit Score* (IFS) measures workload alignment between declared intent and observed runtime behavior. Together, these metrics separate prevention value from system aggressiveness and make system-level evaluation more defensible.

This paper makes four contributions:

- A pre-billing governance architecture that reframes cloud cost control as an early-intervention systems problem rather than a post-billing reporting problem.
- A decision-theoretic intervention engine that weighs predicted waste against the cost of governance actions before execution begins.
- An Intent-Behavior Discrepancy (IBD) framework, operationalized through IFS, for modeling divergence between workload intent and runtime behavior.
- A controlled evaluation benchmark spanning 500 workloads and 28,423 runs that measures pre-billing prevention, runtime governance, anomaly detection, and policy learning under one governance-timing framework.

We evaluate these contributions on a controlled benchmark of 500 workloads and 28,423 runs. The rest of the paper explains why existing systems fail under this timing model, presents the PBCP design as a Prevent → Correct → Learn pipeline, and evaluates whether early intent-aware intervention improves prevention quality over reactive baselines.

2 Motivation

Current cloud governance systems fail for structural reasons, not merely because they lack better telemetry or heuristics. Their failure mode is rooted in when they act, what signals they observe, and whether they can enforce intervention.

2.1 Post-execution blindness

Most cost governance systems observe waste only after execution has already begun or after billing has already closed. Utilization anomalies, idle clusters, and over-provisioned jobs can be reported accurately while still being economically unrecoverable. Once an inefficient workload has consumed

Table 1: Comparison of system categories by intervention timing and governance capability. PBCP is positioned around pre-billing intervention rather than post-billing recommendation.

System Category	Timing	Semantic Intent	Runtime Correction	Pre-Billing Governance
Reactive cloud optimization	Post-billing	No	Limited	No
Utilization anomaly detection	Post-execution	No	No	No
Policy governance systems	Pre-execution	Limited	No	Partial
AIOps systems	Runtime	No	Yes	No
PBCP	**Pre-billing**	**Yes**	**Yes**	**Yes**

hours of compute, the key prevention opportunity has already been lost.

2.2 Semantic ignorance

Reactive systems reason primarily from telemetry. CPU, memory, idle time, and cost spikes are useful signals, but they say little about whether the workload was provisioned consistently with its original intent. A short ad hoc analytics task, a recurring ETL job, and a latency-sensitive streaming service may exhibit overlapping runtime behavior while having very different governance expectations. Systems that ignore intent cannot distinguish expected high-cost behavior from avoidable misalignment early enough.

2.3 Advisory-only governance

Many production tools stop at recommendation. They identify underutilized resources, estimate potential savings, and ask operators to act later. Advisory workflows are valuable for reporting, but they do not close the prevention loop. A system designed for pre-billing control must be able to recommend, block, or auto-correct configurations before cost is incurred, not simply describe waste after the fact.

These limitations motivate a governance model that reasons over workload intent before resource allocation, not only utilization after execution. If governance begins from intent rather than from stale billing evidence, and if likely resource divergence can be estimated before execution, then prevention becomes a systems problem with an enforceable decision point rather than a retrospective analytics problem.

3 Related Work

Prior work on cloud cost control, anomaly detection, and automated operations addresses adjacent parts of the waste problem, but usually leaves the governance decision point after provisioning or after execution. Table 1 summarizes this distinction at the system-category level and frames PBCP as a governance-timing contribution rather than another reporting tool.

3.1 Reactive Cloud Optimization

Reactive cloud optimization systems identify underutilized clusters, idle instances, or inefficient purchasing choices after workloads have already run. These systems are valuable for retrospective cost reduction, but they do not prevent the initial waste event because their evidence arrives after billing has already begun or after execution has already consumed the relevant compute time.

3.2 Utilization-Only Anomaly Detection

Utilization-based anomaly detectors rely on signals such as CPU thresholds, idle time, or abrupt runtime spikes. Such detectors can identify operational abnormalities, but they observe symptoms rather than expected workload behavior. For pre-billing governance, the central question is not only whether utilization is low, but whether the observed behavior is semantically aligned with the declared workload intent.

3.3 Policy Governance Systems

Policy governance systems enforce static constraints such as quota limits, approved instance families, or shutdown requirements. They can block obvious misconfigurations before launch, but they usually lack workload-specific priors and do not reason explicitly about expected savings versus intervention cost. As a result, they remain effective for compliance but limited for adaptive pre-billing governance, intervention-timing optimization, and gaming-resistant prevention evaluation.

3.4 AIOps and Runtime Control Systems

AIOps and autonomous remediation systems provide runtime correction once divergence is visible in telemetry. They are closely related to PBCP's runtime governance stage, but they generally do not infer workload intent before launch or simulate likely waste before billing begins. PBCP differs by combining pre-billing intervention, runtime governance, and policy learning within one intent-aware control loop, and by evaluating prevention with CPS, ESR, and Valid CPS rather than with runtime remediation alone.

4 System Design

PBCP follows a Prevent -¿ Correct -¿ Learn pipeline. The system first infers what the workload is trying to do, estimates whether the request is likely to waste compute, and applies a pre-billing intervention when the expected savings justify the action. Once a workload is running, PBCP continues to monitor for divergence between declared intent and observed behavior, applies runtime corrections when needed, and feeds those outcomes back into future governance decisions.

4.1 Overview

Figure 1 illustrates the full Prevent → Correct → Learn pipeline. A workload enters the system as a natural-language submission, is converted into a structured workload-intent

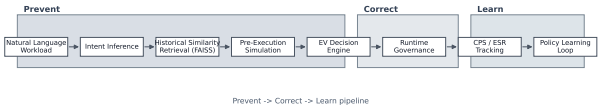

Figure 1: PBCP system architecture. The pipeline groups pre-billing prevention, runtime correction, and policy learning into one governance loop.

representation, is compared with historical analogs, and is then evaluated by a pre-execution simulator and expected-value decision engine before resources are launched.

The figure also shows why PBCP is not only a pre-provision filter. Prevention, runtime governance, and policy learning operate as a continuous control loop so that early intervention, online correction, and later policy adaptation all share the same workload-alignment view.

4.2 Intent Inference

Intent inference converts the submitted description, request metadata, and available team history into a structured workload-intent representation. This representation captures workload type, recurrence, urgency, sensitivity, and resource expectations before execution begins. The purpose is operational rather than descriptive: governance needs an early estimate of what the workload is trying to do so that later decisions are conditioned on expected behavior rather than on telemetry alone.

4.3 Historical Similarity Retrieval

Historical similarity retrieval uses FAISS to query prior workload-intent representations and recover nearby examples with similar execution profiles. These neighbors provide empirical priors for likely utilization, duration, and waste patterns. In effect, retrieval grounds governance in observed workload history rather than relying only on static workload classes. When close matches are unavailable, PBCP falls back to catalog-level priors so that the system remains usable under cold-start conditions.

4.4 Pre-Execution Simulation

The pre-execution simulator combines workload intent with historical priors to estimate likely utilization, duration, cost, and right-sized capacity if the request is executed unchanged. This stage turns workload intent into a concrete governance forecast. Instead of waiting for a job to become visibly inefficient, PBCP estimates whether the requested configuration is already likely to diverge from expected behavior before provisioning occurs.

4.5 EV Decision Engine

The expected-value decision engine compares the outcomes of `BLOCK`, `AUTO_CORRECT`, `SUGGEST`, and `PASS` by balancing predicted prevented cost against intervention cost, delay, and failure risk. This decision-theoretic step matters because governance is not free: aggressive intervention can reduce waste while also increasing execution risk. PBCP therefore treats pre-billing intervention as a cost-sensitive control problem rather than as a fixed rule trigger.

4.6 Runtime Governance

Runtime governance monitors launched workloads for divergence patterns that are not fully observable before execution, including prolonged idle periods, underutilized clusters, and runaway execution. It serves as a second line of defense rather than a replacement for pre-billing intervention. This distinction is important for governance timing: even an accurate simulator cannot eliminate all uncertainty, so runtime correction remains necessary when workload alignment breaks down only after launch.

4.7 Intent-Behavior Discrepancy (IBD)

This stage introduces intent-behavior divergence as the mismatch between declared workload intent and observed runtime behavior. IFS provides the corresponding workload-alignment signal by measuring semantic alignment between intent and behavior. Within PBCP, this signal supports both anomaly diagnosis and later policy learning, making it possible to distinguish low utilization that is expected from low utilization that indicates governance failure.

IFS maps the embedding cosine to four alignment categories: *well-aligned* (IFS ≥ 0.85), *minor divergence* ($0.70 \leq$ IFS < 0.85), *significant divergence* ($0.50 \leq$ IFS < 0.70), and *severe divergence* (IFS < 0.50). These thresholds are chosen so that the IBD flag (IFS < 0.65) falls between the minor and significant bands, giving anomaly detection a conservative operating point that does not require separate threshold calibration.

The score decomposes into four interpretable sub-scores — type alignment, utilization alignment, duration alignment, and resource alignment — weighted 0.35, 0.25, 0.20, and 0.20 respectively. For LLM pipeline workloads, a token-waste sub-score replaces part of the resource term because token budget adherence carries governance significance that node count alone does not capture. These sub-scores are presented in the dashboard as interpretability support rather than as standalone metrics; the canonical IFS value remains the embedding cosine.

The role of IFS within PBCP is diagnostic rather than preventive. A workload can receive high CPS because an effective intervention was applied and high IFS because runtime behavior remained aligned, or low CPS because no intervention was triggered and low IFS because the job behaved contrary to its stated intent. That separation is what makes IFS useful: it explains why waste occurred rather than only measuring whether it was stopped, and it gives the policy learning loop a signal that CPS and ESR do not directly capture.

4.8 Policy Learning Loop

The policy-learning loop closes the system by feeding repeated divergence patterns, missed savings opportunities, and successful interventions back into future governance decisions. Over time, this allows PBCP to move beyond a fixed set of preconfigured rules and improve intervention quality through accumulated workload evidence. In this sense, policy learning is not a separate analytics feature; it is the mechanism that allows governance timing to improve across successive workload generations.

5 Metrics

PBCP is evaluated with a small metric set chosen to reflect operational goals rather than metric diversity. CPS measures prevention value, ESR measures whether prevention preserves useful execution, and IFS measures workload alignment between intent and observed behavior.

5.1 Cost Prevention Score

Cost Prevention Score (CPS) is the primary prevention metric. It measures how much potential waste the system prevents before or during execution relative to the relevant cost exposure. CPS therefore quantifies the benefit of pre-billing intervention and runtime governance in operational rather than purely predictive terms. In this paper, the denominator is always the *unchanged cost exposure*: for pre-provision experiments, the projected cost of executing the submitted request as-is; for runtime experiments, the cost that the same submitted configuration would have incurred without intervention, including any avoidable idle or overrun interval. With that stage-specific denominator, CPS remains bounded by 1.

5.2 Execution Success Rate

Execution Success Rate (ESR) is the anti-gaming constraint. A governance system can inflate prevention metrics by blocking useful work too aggressively, so CPS is only meaningful when interventions still preserve successful workload completion. ESR therefore distinguishes effective governance from trivial waste avoidance through over-restriction. We therefore report *Valid CPS* as CPS $\times$ ESR so that high prevention is only credited when execution remains operationally successful.

5.3 Intent-Fit Score

Intent-Fit Score (IFS) is the diagnostic metric for workload alignment. Within the intent-behavior divergence framing, the canonical IFS definition is the cosine similarity between intent embeddings and behavior embeddings. In this paper, any sub-score decomposition is interpretability support only, not the primary metric definition. The relationship among the metrics is deliberate: CPS indicates how much waste was prevented, ESR indicates whether prevention remained operationally safe, and IFS helps explain why a workload diverged from its expected behavior in the first place.

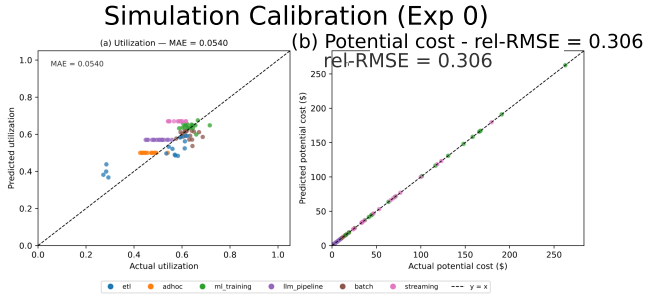

Figure 2: Simulation calibration for Exp 0. Predicted utilization tracks observed utilization closely, with utilization MAE of 0.054; the committed benchmark artifact also reports cost calibration on the same sample.

6 Evaluation

The evaluation asks whether earlier governance timing changes measurable system outcomes. Each experiment targets one operational question: whether workload intent can support accurate pre-execution forecasting, whether pre-billing intervention prevents waste before launch, whether runtime governance remains necessary after launch, whether intent-aware anomaly detection outperforms utilization-only detection, and whether policy learning improves prevention over time.

6.1 Experimental Setup

We evaluate PBCP on a controlled benchmark containing 500 workloads and 28,423 runs across six workload classes. The benchmark includes workload submissions, historical run traces, cost records, runtime anomalies, and intervention stages needed to evaluate pre-billing prevention, runtime governance, and policy learning under one consistent workload-alignment setting.

6.2 Exp 0: Can Simulation Predict Utilization Accurately?

Exp 0 tests whether the simulator is accurate enough to support pre-billing intervention. Figure 2 and Table 2 show utilization MAE of 0.054 and cost rel-RMSE of 0.306. These errors are small enough for governance use even though they are not perfect predictions of future execution. Operationally, the result matters because pre-billing intervention only has value if simulation error is below the threshold at which intervention becomes arbitrary. The remaining cost error is also expected: duration varies at runtime, so a pre-execution model cannot fully eliminate downstream uncertainty.

6.3 Exp 1: Can Pre-Provision Intervention Reduce Waste?

Exp 1 evaluates whether pre-billing intervention prevents waste before resources are launched. Figure 3 highlights the showcase case: PBCP reduces a 20-node ETL request to

Table 2: Summary calibration results for Exp 0. The utilization error remains below the acceptance gate, and cost rel-RMSE stays within the tolerated pre-execution uncertainty range.

Metric	Value	Gate
Utilization MAE	0.054	< 0.10
Cost rel-RMSE	0.306	< 0.40

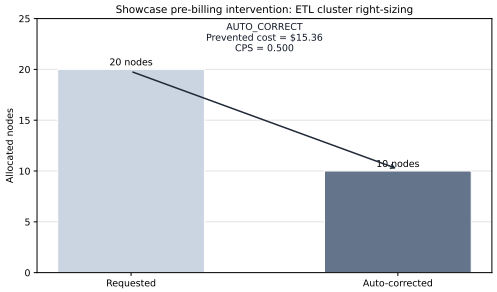

Figure 3: Showcase pre-provision intervention for a 20-node ETL request. PBCP auto-corrects the request to 10 nodes before billing begins, preventing $15.36 with CPS of 0.500.

10 nodes before billing begins, preventing $15.36 with CPS of 0.500. The significance of this result is not the absolute amount of prevented cost in one scenario, but the change in governance timing: the waste path is altered before the cluster is ever provisioned. At the same time, the broader benchmark reports relatively low system-wide CPS because most requests are already near reasonable sizing. This behavior is desirable. A pre-billing governance system should intervene strongly on high-divergence requests without manufacturing unnecessary corrections on already aligned ones.

6.4 Exp 2: Can Runtime Governance Catch Hidden Failures?

Exp 2 evaluates whether runtime governance remains necessary after launch. Figure 4 and Table 3 show that runtime intervention captures failure modes that are only partially visible before execution, including idle persistence, underutilized ETL execution, and runaway ML training. The runaway scenario is especially important: the system prevents $97.92 with CPS of 0.667 by stopping additional cost accumulation after divergence becomes operationally visible. These results suggest that runtime governance is complementary to pre-billing intervention rather than redundant with it; accurate pre-execution simulation reduces avoidable waste, but several divergence patterns still emerge only after workload launch.

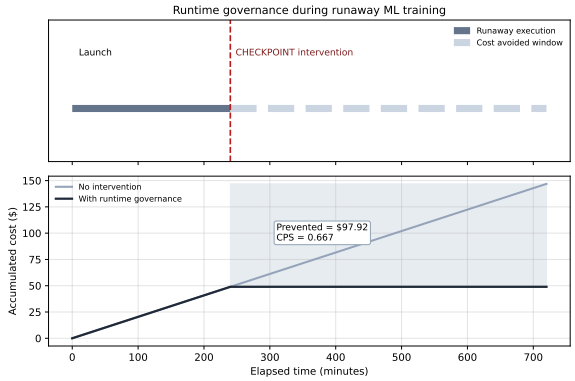

Figure 4: Runtime governance intervention during a runaway ML training workload. The intervention checkpoints prolonged execution at the anomaly boundary, preventing $97.92 of additional cost accumulation.

Table 3: Runtime intervention scenarios from Exp 2. These cases illustrate how runtime governance complements pre-billing control when hidden failures emerge only after launch.

Workload	Intervention	Prevented cost
A Idle adhoc	SCALE_DOWN, TERMINATE	1.61
B Underutil ETL	SCALE_DOWN	21.50
C Runaway ML	CHECKPOINT	97.92

Table 4: Detector metrics for Exp 3. Values are taken directly from the recorded IBD evaluation outputs.

Detector	Precision	Recall	F1
CPU threshold	0.6502	0.5663	0.6054
IFS detector	0.6434	0.9306	0.7608

6.5 Exp 3: Does IFS Outperform CPU-Threshold Detection?

Exp 3 evaluates whether an intent-aware workload-alignment signal improves anomaly detection relative to a utilization-only baseline. Figure 5 and Table 4 show that the IFS-based detector improves F1 from 0.6054 to 0.7608 while substantially increasing recall. The operational implication is that semantic alignment captures divergence missed by simple CPU thresholding, particularly when anomalous behavior is not reducible to a single utilization rule. At the same time, the subgroup comparison is mixed rather than uniformly favorable, so the result should be interpreted as evidence that intent-aware detection is stronger overall, not as proof that every workload subgroup behaves identically.

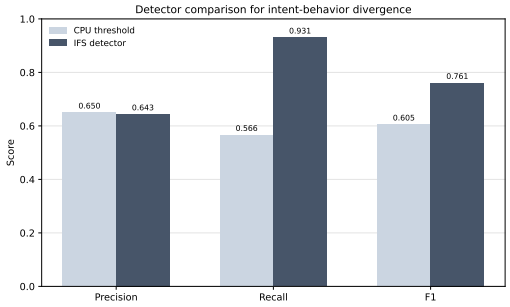

Figure 5: Detector comparison for Exp 3. The IFS-based detector improves overall detection quality relative to the CPU-threshold baseline on the committed benchmark outputs.

The F1 improvement from 0.605 to 0.761 is driven primarily by recall. The IFS-based detector captures anomalous workloads that the CPU-threshold baseline classifies as normal because their utilization does not breach the detection boundary. This pattern is consistent with the design intent of IFS: a workload that finishes early and idles, consumes fewer resources than requested, or executes a materially different computation than its description implies can remain below the utilization alert threshold while still producing measurable intent-behavior divergence.

The mismatch-subgroup result adds an important qualification. Among workloads flagged as type-mismatched by intent inference, anomaly rates are higher and mean IFS is lower than in the non-mismatch group, consistent with the hypothesis that declared-to-inferred type divergence predicts runtime divergence. However, the anomaly rate difference across subgroups is not large enough for type mismatch alone to serve as a reliable detection signal. The subgroup result supports IFS as a composite detector rather than suggesting that any single structural feature is sufficient.

The precision tradeoff is also worth stating directly. IFS precision on this benchmark is lower than the CPU-threshold baseline, meaning intent-aware detection produces more false positives per true positive. This is a consequence of IFS operating at broader sensitivity: it flags misaligned workloads earlier and more broadly than a utilization gate would. The threshold sweep shows that an operator can recover precision by raising θ_{ifs} at the cost of recall, which provides a practical tuning handle once deployed against a production workload population.

6.6 Exp 5: What Is Overall System Effectiveness?

Exp 5 provides a system-level roll-up using CPS, ESR, and IFS together. This combination matters because prevention quality, execution safety, and workload alignment answer different questions. CPS alone can overstate success if a system avoids cost by over-blocking workloads, while ESR alone cannot indicate whether meaningful waste was prevented. IFS adds diagnostic structure by showing whether low-cost outcomes are associated with aligned workload behavior rather than with arbitrary restriction. In the recorded benchmark, PBCP reaches Valid CPS of 0.5585 with ESR of 0.9809, indicating that prevention remains high even after discounting unsuccessful or overly aggressive intervention behavior. Taken together, the roll-up suggests that the system can improve economic prevention while preserving useful execution and maintaining meaningful workload alignment.

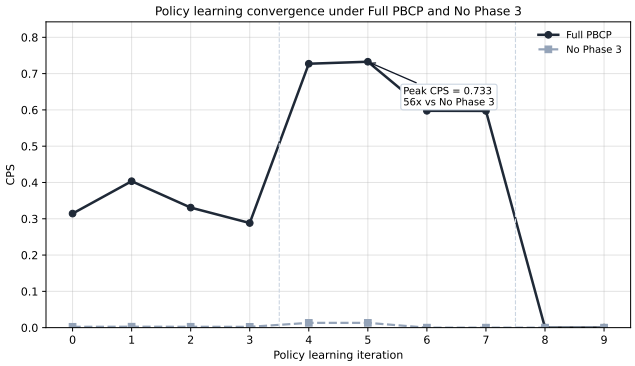

Figure 6: Policy-learning convergence in Exp 6. Full PBCP reaches peak CPS of 0.733, compared with 0.013 for the No Phase 3 baseline, demonstrating a 56× improvement at peak.

Table 5: Available Exp 6 ablations from the committed benchmark outputs. Peak CPS is reported for each variant without relabeling unsupported baselines.

Variant	Peak CPS	Peak generation
Full PBCP	0.733	5
No Phase 3	0.013	4
Policy-only	0.000	0
Embedding-only	0.109	2

6.7 Exp 6: Does Policy Learning Improve Over Time?

Exp 6 evaluates whether policy learning improves governance quality over time. Figure 6 and Table 5 show that Full PBCP reaches peak CPS of 0.733, whereas the No Phase 3 baseline peaks at 0.013. This 56× separation is the strongest evidence that the learning loop changes system behavior rather than merely documenting it. The ablation comparison also clarifies what is being learned: static policy-only control contributes little in this setting, and embedding-only retrieval is useful but insufficient by itself. Governance quality improves most when workload alignment, intervention policy, and feedback are integrated into one loop rather than deployed as isolated components.

7 Discussion

The evaluation shows that earlier governance timing can improve prevention, but it does not establish that PBCP is immediately ready for production enforcement. The discussion therefore focuses on the boundary between what the benchmark supports today and what would still need validation in operational settings.

7.1 Limitations

Although the benchmark captures diverse workload behaviors, it remains a controlled synthetic environment rather than production telemetry. Real enterprise workloads may exhibit additional operational variance, organizational approval constraints, and intervention costs not represented here. The current results also depend on assumed policy and intervention parameters, the policy learning loop is still evaluated at limited scale, and the system has not yet been validated inside a live enterprise enforcement stack. Similarly, the Exp 3 mismatch-subgroup result is mixed even though overall detector quality improves, so the current anomaly-detection claim should be read as strong benchmark evidence rather than as final production generalization.

7.2 Future Work

The most direct next steps are calibration against real organizational telemetry, integration with execution platforms such as Databricks or Kubernetes, stronger online policy learning, and tighter coupling with production approval and enforcement stacks. These extensions would test whether the same workload-alignment signals remain useful once governance decisions interact with real operators, heterogeneous environments, and organization-specific risk tolerances.

7.3 Anomaly Root Cause Analysis and Prevention Feedback

The RCA module converts recurring divergence patterns into policy suggestions, closing the governance loop between runtime monitoring and future pre-billing intervention. After each generation, `RootCauseAnalyzer` queries historical incidents grouped by workload type and incident category — over-provisioned requests, idle cluster patterns, runaway execution, and undeclared token budget — and synthesizes a `Policy` object when the same pattern recurs at least three times within a lookback window. Confidence scales with pattern frequency up to a ceiling of 0.95, and only policies above the 0.60 threshold enter the registry.

This mechanism addresses a gap that static pre-billing policy cannot fill. Preconfigured rules handle known divergence patterns, but novel or low-frequency patterns are not covered at system initialization. The feedback loop fills that gap incrementally: workloads that escape pre-billing intervention, behave anomalously at runtime, and are flagged by IFS contribute to learned policies that catch the same pattern in subsequent submissions. The Exp 6 convergence result is partly attributable to this accumulation — Full PBCP substantially outperforms the No Phase 3 ablation because learned policies progressively reduce the baseline rate of uncaught divergence.

Two limitations apply to the current implementation. First, the feedback loop operates on synthetic incident data, so confidence and frequency estimates may not reflect the noisier incident streams in real enterprise environments. Second, the pattern-counting logic does not weight incidents by recency or cost severity, meaning a cluster of low-cost incidents and a smaller cluster of high-cost incidents are treated identically. Both limitations are addressable in a production deployment and represent near-term extensions rather than design defects.

8 Conclusion

Cloud governance systems frequently act too late to prevent the waste they later diagnose. PBCP addresses this governance-timing gap by combining workload-intent inference, historical similarity retrieval, pre-execution simulation, decision-theoretic intervention, runtime governance, and policy learning within one workload-alignment framework. Across the current benchmark, PBCP shows that pre-billing intervention can prevent waste before launch, runtime governance can still correct post-launch divergence, and policy learning can materially improve prevention quality over time. The broader implication is that cloud cost control should be treated as an early-intervention systems problem rather than as a purely post-billing optimization task.

References